程序设计原理

指针2

任课教师： 喻　梅
于瑞国
王建荣
李雪威
赵满坤

Void Pointers

- The void type of pointer is a special type of pointer. In C++, void represents the absence of type, so void pointers are pointers that point to a value that has no type (and thus also an undetermined length and undetermined dereference properties).
- This allows void pointers to point to any data type, from an integer value or a float to a string of characters.
- But in exchange they have a great limitation: the data pointed by them cannot be directly dereferenced (which is logical, since we have no type to dereference to), and for that reason we will always have to cast the address in the void pointer to some other pointer type that points to a concrete data type before dereferencing it.

Void Pointers

```
#include <iostream>
using namespace std;
void increase (void* data, int psize) {
      if ( psize == sizeof(char) ) {
            char* pchar;
            pchar=(char*)data;
            ++(*pchar);
      }
      else if (psize == sizeof(int) ) {
            int* pint;
            pint=(int*)data;
            ++(*pint);
      }
}
int main () {
char a = 'x';
int b = 1602;
increase (&a,sizeof(a)); increase (&b,sizeof(b));
cout << a << ", " << b << endl;
return 0;
}
```

Pointer in Function

- **Pointer as parameter**
  - **The address stored in the pointer is passed to the function**
  - **A kind of pass-by-reference**

```
void swap(int *p1, int *p2)
{           int temp;
            temp = *p1;
            *p1      = *p2;
            *p2      = temp;
}
```

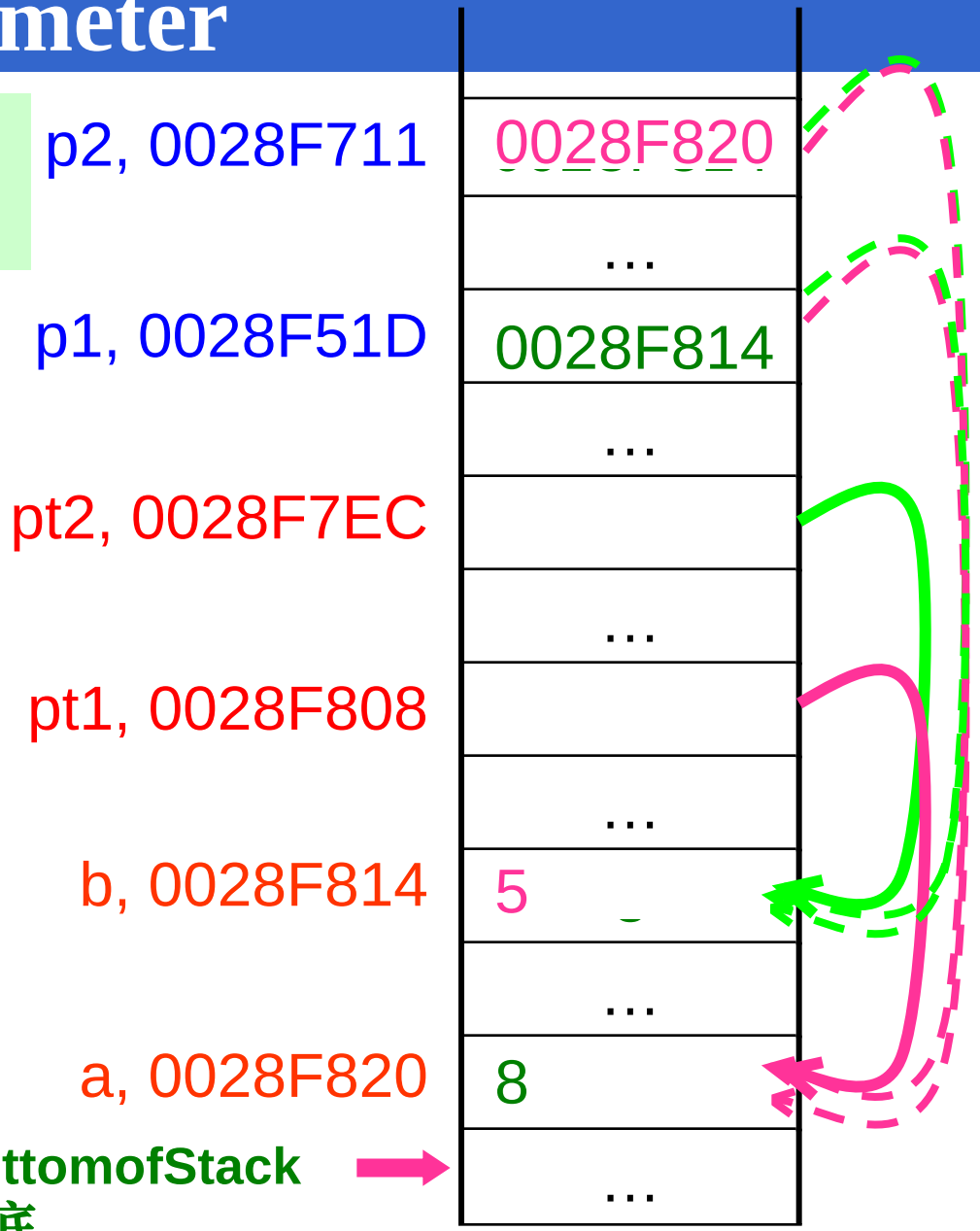
```
void swap(int *p1, int *p2)
{           int *temp;
            temp = p1;
            p1 = p2;
            p2 = temp;
}
```

Pointer as parameter

对指针本身来讲，是“值传递”；
对指针指向的目标来讲，是“引用传递”。

```
void myswap(int *p1, int *p2){
        int *temp;
        temp = p1;
        p1 = p2;
        p2 = temp;
}
```

```
main(){
   int *pt1, *pt2;
   …
   myswap(pt1, pt2);
   …
}
```



Pointer as Return Value

- **The address stored in a pointer is returned**

```
int max=0;
int *getMax();

void main(){
    int *p;
    p=getMax();
    cout<<"Max is "<<(*p);
}
```

```
int *getMax(){
    int tmp=max, i;

    for(i=0;i<3;i++){
        cout<<"Please enter No. "<<i<<endl;
        cin>>tmp;
        if(tmp>max) max=tmp;
    }
    return &max;
}
```

Don't return a local pointer!
--it becomes invalid after the return!

Passing Arrays to Functions

```
double getAverage(int arr[], int
size);
int main ()
{
   int balance[5] = {1000, 2, 3, 17,
50};
   double avg;
   cout << balance << endl;
   cout << &balance << endl;
   cout << &balance[1]<< endl;
   avg = getAverage( balance, 5 ) ;

   cout << "平均值是：  " << avg <<
endl;

   return 0;
```

```
double getAverage(int arr[], int
                                    g;
```

```
0x5ffe80
0x5ffe80
0x5ffe84
0x5ffe80
0x5ffe60
0x5ffe80
0x5ffe84
平均值是： 214.4
```

```
   return avg;
}
```

Dynamic Memory Allocation

- The method allowing programmers to allocate storage
  - Dynamically (during the execution time) and
  - "Manually" (using explicit statement)
- "new" operator: to allocate memory

```
int *pValue = new int;

int *mylist = new int[10];
```

Allocate memory for an int variable;
The address is assigned to "pValue".

Allocate memory for an array of 10 int elements;
The address is assigned to "mylist".

The “delete” Operator

- **The memory allocated by “new” remains available until you explicitly free it or the program terminates.**

It’s good manner to always explicitly free memory allocated by “new”.

- **“delete”: to free memory reserved by “new”.**

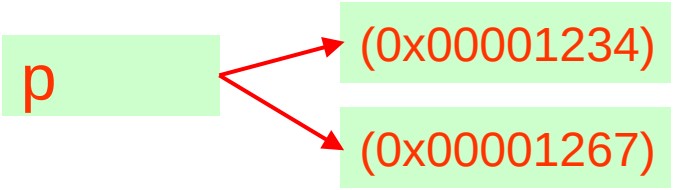
```
delete pValue;
delete [] mylist;
```

- **Memory leak**

  The memory space becomes inaccessible due to the loss of pointing

```
int *p = new int;
p = new int;
```

delete p;



Multidimensional Dynamic Arrays

- **To create a 3x4 multidimensional dynamic array**
  - **View multidimensional arrays as arrays of arrays**
  - **First create a one-dimensional dynamic array**
    - **Start with a new definition:**

```
typedef int* IntArrayPtr;
```

    - **Now create a dynamic array of pointers named m:**

```
IntArrayPtr *m = new IntArrayPtr[4];
```

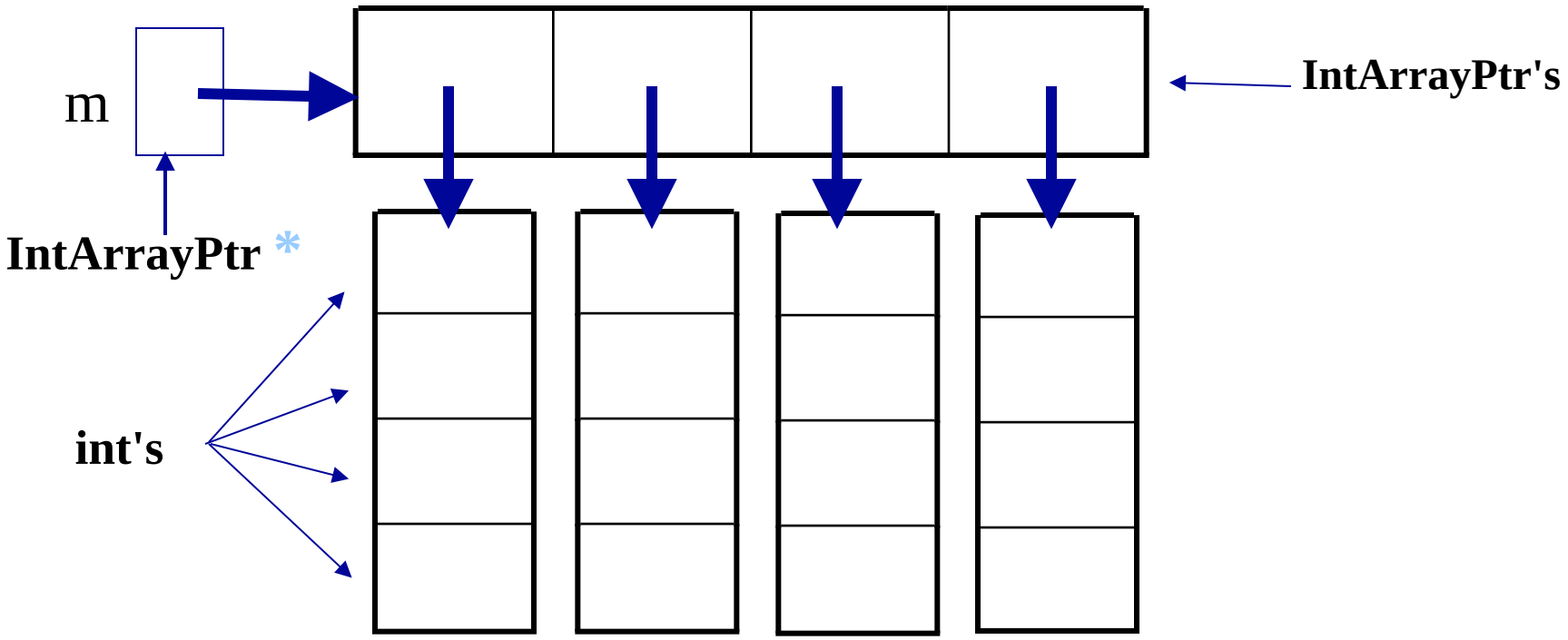
  - **For each pointer in m, create a dynamic array of int's**

```
for (int i = 0; i<4; i++)
    m[i] = new int[4];
```

```
int **m;
m=new int *[4];
for(int i=0;i<4;i++)
   m[i]= new int[4];
```

A Multidimensial Dynamic Array

- **The dynamic array created on the previous slide could be visualized like this:**



Exercise

- **指针练习:是否存在该字符**

  编写程序，从键盘输入一个字符ch，在字符串中查找是否存在有该字符，若存在，则给出该字符在字符串中第1次出现的位置。

  要求设计函数char * search(char *str,char c)，其功能为：在str所指的字符串中，查找是否有字符变量c的字符，如果有，返回字符串中相同字符的地址，如果没有则返回NULL

- 输入：分别输入字符串str和字符c
- 输出：如果有则输出yes,没有则输出no.

- 样例输入：

  Ashdaslas
  h
- 样例输出：

  yes

Exercise

- **指针练习：报文解密**

  输入一个字符串，将该字符串进行解密。解密规则：大写字母‘A’, ‘B’,‘C’, ‘D’用‘W’, ‘X’, ‘Y’, ‘Z’代替，对于‘E’到‘Z’的字母，分别用原字母的前4个字符代替，其他字符保持不变。
  （要求使用指针实现）

- 输入：输入字符串str
- 输出：解密后的字符串.

- 样例输入：
  ABCDEFGHIJKLMNOPQRSTUVWXYZ
- 样例输出：
  WXYZABCDEFGHIJKLMNOPQRSTUV

Exercise

- 指针练习：矩阵加法

  求两个n*n（n<10）矩阵的加法。矩阵的数据由输入决定，输出矩阵和。 实现函数

  void ADD(int(*pa)[10],int(*pb)[10],const int n)

  将行指针pa指向的矩阵与行指针pb指向的矩阵相加，结果存放到pa指向的矩阵中。

- 输入：

第一行输入n的数值；
然后输入矩阵a的n*n个数据和矩阵b的n*n个数据。

- 输出：

矩阵a和矩阵b相加后得到的矩阵。

- 样例输入：

```
3
1 2 3
4 5 6
7 8 9
7 8 9
4 5 6
1 2 3
```

- 样例输出：

```
8 10 12
8 10 12
8 10 12
```